ENS v2 Design Doc

1. Introduction

1.1 Problem Statement

ENS has been massively successful as a self-sovereign naming system for distributed systems. Since its inception in 2017, the Ethereum platform, and the wider ecosystem around it, have developed substantially, and ENS has had to adapt to meet new challenges.

One of the major developments since ENS launched is the rise of L2s, and the expectation that user activity will shift from L1 Ethereum transactions and onto L2s, in order to offer improved scalability and lower transaction fees to users.

With the development of CCIP-Read, and infrastructure that builds upon this such as the EVM Gateway, ENS has enabled users to delegate resolution of their names to external systems, including L2s, as well as external databases and centralised services. This flexibility has been put to good use with a number of high profile integrations including cb.id, uni.eth and dao.eth.

However, ultimately scalability and usability concerns necessitate moving registration, renewal, and updates of .eth second-level names (2LDs), along with setting of primary names, from Ethereum to an L2. This will naturally require a substantial migration, as well as updates to clients that resolve names and other ENS-related infrastructure. While this will be a significant technical challenge, it also provides us with a unique opportunity to take a fresh look at ENS and rearchitect it in a way that best serves the needs of its users as we understand them now, with 7 years of experience.

1.2 Goals of the New Version

ENS has always had a few core goals that inform the design, structure, and governance of the system:

- **Preserve user sovereignty**: Users should truly own their names, and be free from outside interference with that ownership.
- **Maximise utility**: ENS is built as a naming system, designed to improve the usability of crypto applications. When tradeoffs have to be made, prioritize end users and their experience.

- **Stability of ownership**: All else being equal, a user should be able to assume that a name that resolves one way today will still resolve the same way tomorrow.
- **Minimise surprise**: The intuitive expectation is usually the correct one. Design the system to minimise unexpected outcomes.
- **Flexibility**: Wherever possible without compromising the goals already stated, maximise the flexibility of the system. Use indirection and introspection wherever possible to provide for functionality improvements without the risks that come from centralised upgrade mechanisms.
- **Offchain Indexing**: It must be possible to reconstruct an authoritative record of name ownership and resolver addresses by indexing events emitted by the contracts concerned. This requirement does not extend to being able to index the values names resolve to, as dynamic resolvers and CCIP-Read present situations in which this is not possible.

To this we can add some new goals for v2:

- **Preserve existing guarantees:** Name ownership under the new system should be the same as under v1. Where new tradeoffs are available (for example, accepting the weaker security guarantees of an L2 in order to benefit from reduced transaction fees), provide users with a way to choose to either preserve their existing guarantees or benefit from the new ones.
- **Preserve backward compatibility:** Minimise changes that directly affect users. Where those changes have to be made, minimise the impact of the changes. Change that directly affects end users is more important to avoid than change that affects developers.
- **Enhance scalability**: The new system should reduce transaction fees and increase maximum throughput for users by a significant factor.
- **No favoured-nation status:** Although .eth names will be moved to a singular L2, this shouldn't imply any special status for that L2 over others; users should still be able to use CCIP-Read and other mechanisms to delegate names as desired, with no additional overhead compared to the L2 used by .eth.

2. Current System

2.1 Overview of ENS v1

The ENS system comprises three main parts:

- The ENS registry
- Resolvers
- Registrars

The registry is a single contract that provides a mapping from any registered name to the resolver responsible for it, and permits the owner of a name to set the resolver address, and to create subdomains, potentially with different owners to the parent domain.

Resolvers are responsible for performing resource lookups for a name - for instance, returning a contract address or a content hash as appropriate. The resolver specification defines what methods a resolver may implement to support resolving different types of records.

Registrars are responsible for allocating domain names to users of the system, and are the only entities capable of updating the registry; the owner of a node in the ENS registry is its registrar. Registrars may be contracts or externally owned accounts, though the root and top-level registrars, at a minimum, are implemented as contracts.

Resolving a name in ENS is a two-step process. First, the ENS registry is called with the name to resolve. If the record exists, the registry returns the address of its resolver. If the record does not exist, the parent of the name is tried, and so on recursively until a resolver is found. Then, the resolver is called, using the method appropriate to the resource being requested. The resolver then returns the desired result.

CCIP-Read extends this mechanism, by allowing resolvers to request additional information from offchain sources in order to complete the resolution process; using this mechanism, names can be resolved using data from L2s or other systems, verified against data committed to Ethereum.

2.2 Identified problems and limitations

- CCIP-Read permits delegating subtrees of ENS to other systems, but the .eth registrar remains on L1, which means that registering and renewing names will always require L1 gas fees. Although we anticipate that in the long run, most end-users will own subnames rather than .eth 2LDs, we still

expect .eth registrations and renewals to grow over time, and it is important that gas costs relating to .eth registration and renewal remain affordable.

- Delegating an ENS name to an L2 or offchain system still requires at least one L1 transaction to set the resolver, imposing additional L1 gas costs on users who want to manage their names offchain.
- A number of improvements in the ownership model have been identified and implemented via the Name Wrapper. This enables new functionality, but at the cost of additional overhead and indirection. This functionality could be incorporated directly into the ownership model of a new registry.
- The ENS registry has no understanding of name expiration, and so when a .eth name expires, it continues to exist in the registry until replaced by a new registration. Ideally the ownership model should allow flexible ways to modify the ownership and resolution of names.
- Because the registry is flat, there is no way to replace or invalidate an entire subtree when the owner of the parent name wishes to or when the name expires - subnames have to be individually deleted or updated.

3. Proposed System (ENS v2)

3.1 Summary

A new design for the ENS registry is proposed, changing from a single flat registry to hierarchical registry contracts. Each name has its own registry, which manages subdomains and the resolver for the name.

This requires changes in the resolution process, which will be streamlined by releasing an officially supported Universal Resolver that targets the current ENS system, and transitioning clients over to using this in favor of resolving directly. When ENSv2 is deployed, clients can be updated to reference a new universal resolver implementation, making the transition to ENSv2 simple for clients and DApps.

A new deployment on an L2 - either a public one or a dedicated chain - is created to act as the authoritative registry for .eth names. When migrating existing names, users can choose to either keep their name on L1, or move it to the new L2; registrations and renewals will always occur on the L2. Names can be moved between the L2 and L1, with L1 names incurring L1 gas fees for

updates, but having the strongest security and availability guarantees, as well as improved latency.

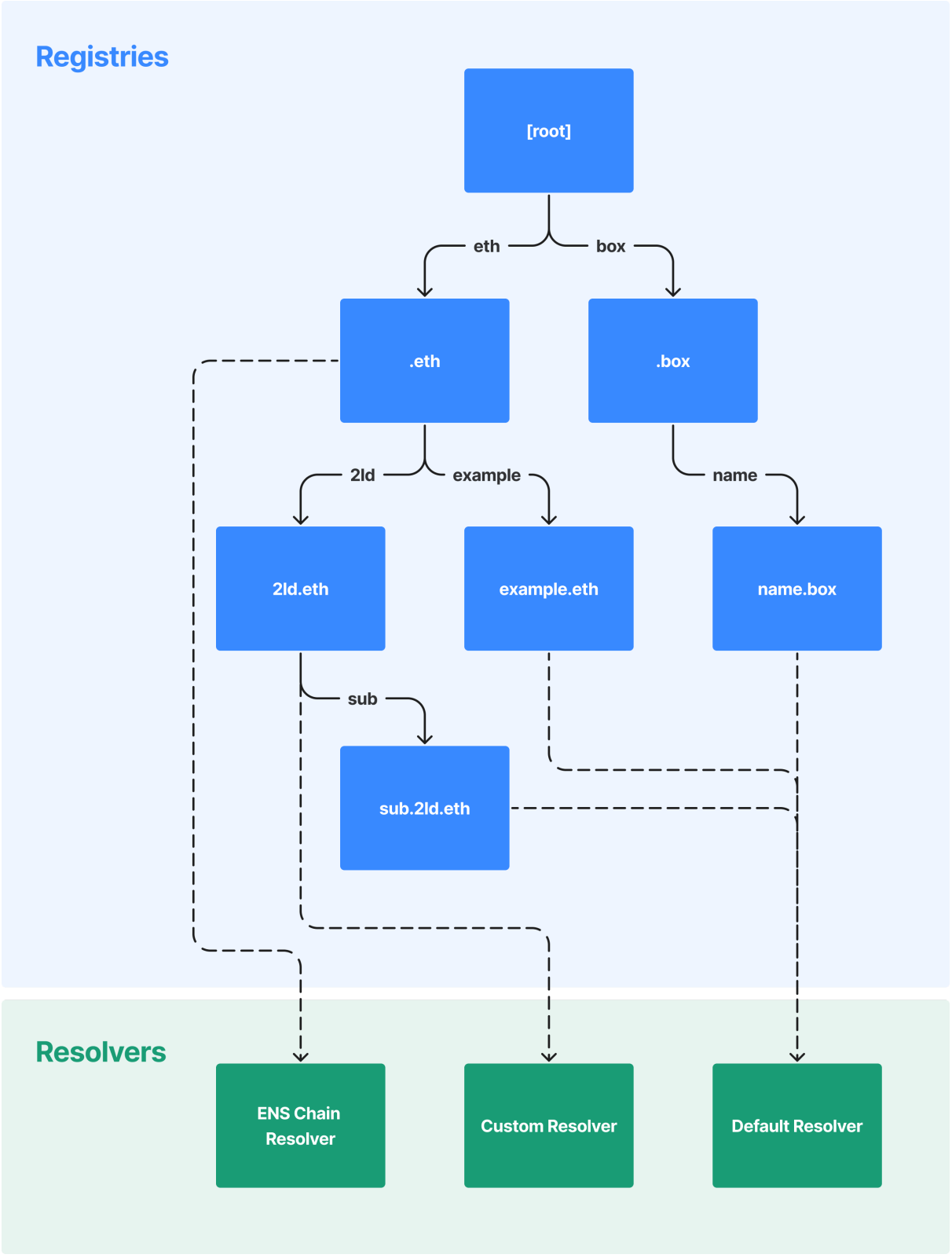
Names hosted on the L2 are resolved by means of a CCIP-Read gateway and compatible resolver, which fetches resolution information from the L2 trustlessly.

3.2 Overview

3.2.1 Registries

The singular flat ENS registry is replaced with a hierarchical registry, with each registry responsible for one name and all its direct subnames. The registry acts as an NFT collection for all the subnames of the name it is responsible for. A root registry, containing the TLDs, serves as the entry point to the system.

In addition to ERC1155, each registry implements a standard interface, with methods for fetching the address of the registry responsible for a subname, and for fetching a subname’s resolver address.



The resolution process is modified so that the resolver address is fetched by recursively querying registries to find the one closest to the target name, then querying that registry for its resolver. The resolution process thereafter is unmodified from v1.

3.2.2 Universal Resolver

A universal resolver contract provides for easy one-step resolution of names by any CCIP-Read compatible client; clients simply call a ‘resolve’ method and the universal resolver handles all resolution logic internally. This contract, rather than the root registry, serves as the entrypoint and sole hard-coded address for most clients.

Resolving via a universal resolver also provides an easy migration path for clients: a universal resolver that targets the current ENS implementation will be deployed, and clients migrated over to using it instead of direct resolution, prior to migration to ENS v2.

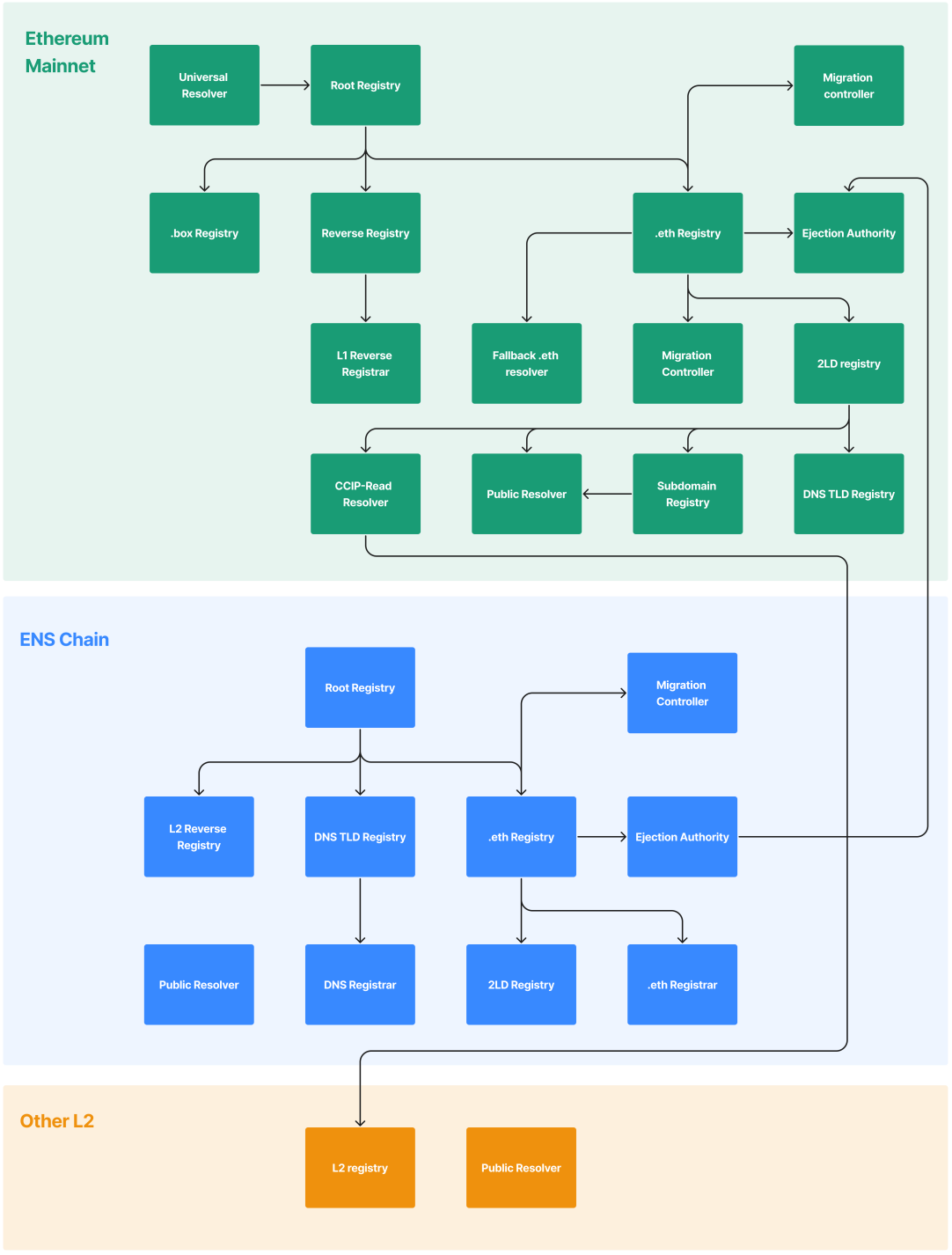
3.2.3 .eth and L2

A new .eth registry is developed for deployment on an L2 network. This registry supports existing ENS functionality using the new registry architecture, alongside the ability to move .eth names between L2 and L1. All .eth names are registered on the selected L2, but once registered may be moved to L1 so they can be resolved without additional overhead via CCIP-Read; this process is intended for users wishing to delegate their names to other L2s (eg, for subname issuance) or for users who are particularly sensitive to latency or security concerns.

An L1 counterpart to the new L2 .eth registry is developed that supports the functionality provided by the L2 .eth registry.

3.3 Deployment Architecture

The core ENS deployment architecture consists of parallel registry hierarchies on L1 and the selected L2, with communication between them via cross-chain messaging:



3.4 Migration Plan

All contracts will be initially deployed to both L1 and L2 with temporary permissions as needed - eg, registrations and renewals will be disabled, contracts will be owned by an EOA used for the migration, and this account will also be authorised as a controller on the .eth registry in order to register names

directly. The initial deployment will include the root registry, TLD registries, and associated utility contracts such as the Universal Resolver and .eth registry controllers.

After deployment, an initial sync will be performed, creating L2 entries for all .eth 2LDs, and transferring ownership of these entries to a migration contract. This contract will facilitate the migration process for .eth 2LD owners who wish to move their names to L2, and ensures that these names cannot have duplicate registrations on L2.

On upgrade day, the following sequence of operations takes place:

1. Disable new registrations and renewals on ENSv1.
2. Perform a new sync, updating the state of ENSv2 on L2 to match the set of registrations and expiries existing on L1.
3. Switch over clients to using the new universal resolver for ENSv2, possibly using a one-time-upgradeable proxy to allow this to be done instantaneously.
4. Enable registrations and renewals on ENSv2.

Afterwards, users can migrate their names to ENSv2, either to L1 or to L2. The migration process, described in more detail below, entails transferring the ENSv1 name to a system contract, which then creates or transfers the ENSv2 name to the user on either L1 or L2. The L1 .eth registry and other contracts have functionality to ensure that unmigrated names continue to resolve on ENSv2 as they did on ENSv1, and even changes to the ENSv1 registry will continue to be reflected in ENSv2 for unmigrated names.

Migrating names that use the name wrapper and are locked or emancipated requires particular care. This is achieved by requiring that the migrated name use a specific registry on L1 or L2, which enforces the same restrictions as the name wrapper did on ENSv1. Migration of subnames (3LD and below) also requires particular care and is discussed in more detail in section 4.

3.5 Improvements over ENS v1

The most significant change from ENS v1 is the transition from a single flat registry, to a directional graph of registries. While the flat registry combined with namehash hashing permitted an extremely straightforward implementation, it limited all names to the same ownership model, enforced by the single central registry contract.

Transitioning to a graph-structured registry permits each name to define its own ownership and transfer model, so long as it adheres to the expected standard interfaces. This means that the .eth registry can implement name expiration directly, meaning that when a name expires, it vanishes from the registry and ceases being resolvable immediately.

Further advantages include:

- Entire subtrees can be recursively replaced by updating the registry for a parent name. This makes replacing or deleting multiple names significantly more efficient, and makes it possible for expired or transferred names to start from a ‘blank slate’.
- Functionality that was previously handled by registrar contracts - contracts that own registry entries - can now be incorporated directly into the registry implementation.
- Users can define their own registry contracts with custom functionality, such as custom expiration and transfer rules. Combined with the ability to ‘lock’ the registry for a name, this can facilitate much of the functionality of the name wrapper, while still permitting significantly more flexibility.
- Standardised implementations for users can be provided and used by most end-users, while still permitting advanced users to choose their own - similar to the way that the Public Resolver functions today.

4. Technical Details

4.1 Terminology

Registry: A contract that is responsible for maintaining information about a name and its direct subnames. Registries store the address of the resolver for the name they are responsible for, along with the addresses of the owner and subregistry for each direct subname of that name. A registry is always directly responsible for only one name. Registries do **not** administer ownership information for names they are responsible for - that is the job of the parent registry. All registries are ERC1155 token contracts.

Shadow Registry: A registry entry and registry owned by a system contract. Shadow entries exist in order to hold a place in the registry hierarchy for a name held on another chain (L1 vs L2). For example, shadow entries are

created for unmigrated locked .eth names in order to allow owners of subdomains to migrate independently of their parent names.

4.1 Invariants

1. Every name in the registry tree is rooted at a single root node.
2. Registries control access to resolver and subregistry addresses for their entries, and can enforce restrictions on how they are updated. For example, a name's subregistry can be locked. Once a subregistry is locked, it can no longer be changed by the owner of the name or the owner of the parent name.
3. Legacy ENS names must continue to function on L1 until migrated by the owner. This includes creation, update, and deletion of new names in the legacy ENS registry.
4. The same name cannot be owned by a user-defined address on both L1 and L2.
5. The same name cannot have user-defined resolvers on both L1 and L2.

4.3 Architecture

4.3.1 IRegistry

The core component of ENSv2 is the registry interface. All registries must implement the following interface:

```
interface IRegistry is IERC1155 {
    event NewSubname(string label);

    function getSubregistry(string calldata label) external v
    function getResolver(string calldata label) external view
}
```

`getSubregistry` is passed a label, and is expected to return the address of the registry responsible for the label, or `address(0)` if none exists.

`getResolver` is passed a label, and is expected to return the address of the resolver for the specified label, or `address(0)` if none exists.

Registries must **not** query subregistries to satisfy calls to `getSubregistry` or `getResolver`; all queries are answered based on information held by the registry itself.

The `NewSubname` event **must** be emitted whenever a new subname is minted, containing the plaintext label for the new subname.

4.3.2 RegistryDatastore

Registries are expected to store key resolution information in a single universal storage contract, the datastore. This facilitates lookups via CCIP-Read for L2 deployments, by reducing the number of distinct accounts whose storage roots need proving.

The interface for the `RegistryDatastore` is as follows:

```
interface IRegistryDatastore {
    event SubregistryUpdate(address indexed registry, uint256
    event ResolverUpdate(address indexed registry, uint256 in

    function getSubregistry(address registry, uint256 tokenId
    function getSubregistry(uint256 tokenId) external view re
    function getResolver(address registry, uint256 tokenId) e
    function getResolver(uint256 tokenId) external view retur
    function setSubregistry(uint256 tokenId, address subregis
    function setResolver(uint256 tokenId, address resolver, u
}
```

The `RegistryDatastore` is deployed as part of the initial ENS deployment, and every registry is expected to have an immutable reference to it. Registries **may** store other data, such as ownership information, in their internal contract storage, but **must** store subregistry and resolver information in the datastore. The `flags` fields can be used for implementation-specific data on each name, and registries **should** use these fields to store any additional information required to resolve names - for example, expiration timestamps.

4.3.3 Resolvers

The interface and requirements of resolvers remain unchanged from the requirements specified in ERC-137 and ENSIP-10.

4.3.4 Universal Resolver

The universal resolver implements the resolution process, described below, as an EVM contract. This simplifies client implementations significantly, as they only need to be able to call a standardized interface on the universal resolver and handle CCIP-Read callbacks; there is no need for each client to understand and implement the complete ENS resolution process. This also makes future upgrades to the resolution process much more straightforward, requiring only that clients update the address for the universal resolver implementation they use.

4.4 Resolution Process

The ENSv2 resolution process consists of two steps: finding the resolver address, and querying the resolver.

Finding the Resolver Address

Let `head(name)` be a function that returns the first label of `name` and `tail(name)` be a function that returns the remaining labels. If `name` is a single label, `head()` returns that label, and `tail()` returns the empty string.

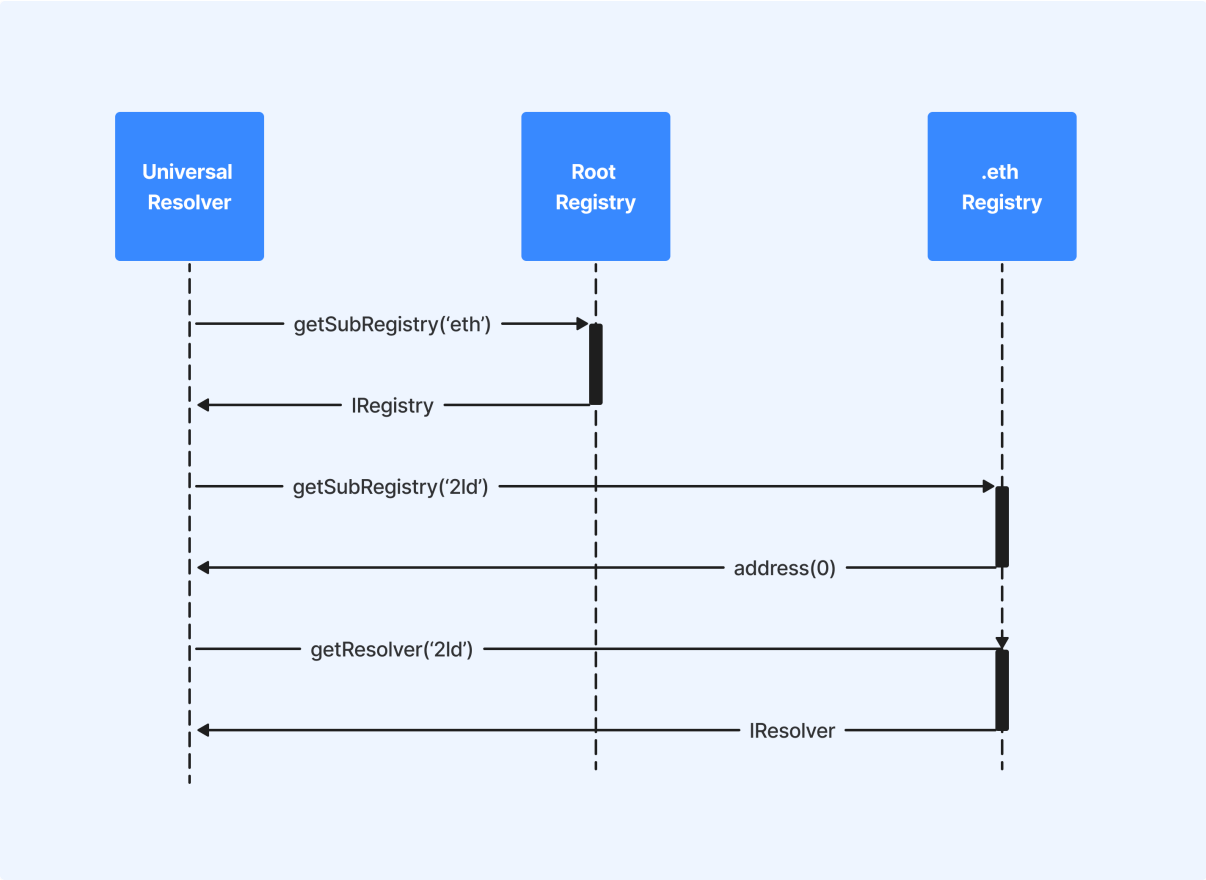
Let `||` be an operator over two addresses that returns the first address that is not `address(0)`, or `address(0)` if both addresses are 0.

The process to find the resolver responsible for `name` is thus defined by the following pseudocode:

```
def findResolver(name) -> (IRegistry, IResolver):
  label = head(name)
  rest = tail(name)
  if rest == '':
    return (root.getSubregistry(label), root.getResolver(labe
  registry, resolver = findResolver(rest)
  if registry != address(0):
    subregistry = registry.getSubregistry(label)
    subresolver = registry.getResolver(label)
    return (subregistry, subresolver || resolver)
  return (address(0), resolver)
```

Intuitively, this finds the longest suffix of `name` that has a resolver set, and returns it.

Example



In this example, `2ld.eth` exists in the .eth registry, and has a resolver set, but no subregistry. We are attempting to resolve `foo.2ld.eth`.

1. We call `getSubregistry('eth')` on the root registry. It returns the address of the registry responsible for `.eth`.
2. We call `getSubregistry('2ld')` on the .eth registry. Since `2ld` does not have a subregistry set, it returns `address(0)`.
3. We call `getResolver('2ld')` on the .eth registry. It returns the address of the resolver responsible for `2ld.eth` (and thus any wildcard subdomains).

Querying the Resolver

Querying the resolver follows the same process described in EIP 137 and ENSIP 10.

L2 Resolution

As with ENSv1, resolvers may revert with CCIP-read lookup requests, which **must** be satisfied by the client. These may take any form, but a standardized implementation is described in *4.5.5* that implements a parallel resolution process on L2 using CCIP-Read, storage proofs, and the `RegistryDatastore` contract.

4.5 Deployment

4.5.1 Root Registry

The ENS naming hierarchy starts at the root registry. The root registry is responsible for storing and administering all the top-level domains (TLDs) in ENS. The root registry implements an ownership model, with its owner able to create and replace TLDs as needed. TLDs can also be 'locked', which prohibits any further changes in their subregistry address.

The same root registry is used on L1 and L2.

4.5.2 L2 .eth Registry

A new registry for .eth names on L2 will be the authoritative source for information on .eth registrations. All .eth registrations from L1 will be migrated to this new registry as part of the transition to ENSv2.

The registry will utilise the same controller pattern used by the current one, where core invariants such as name ownership are enforced by the registry, and controller contracts are responsible for handling registration and renewal logic, payments, etc. This allows the core registry contract to be locked, while still providing for upgradeability over other parameters.

Key registry functionality includes:

- Allowing users to register new .eth names
- Allowing users to register recently expired names via the premium process
- Processing renewals of .eth names
- "Ejecting" names to L1
- Allowing .eth 2LD owners to set a subregistry for their name

Ejecting a name to L1 is handled in conjunction with the L1 .eth registry and ejection controller contracts, documented below.

4.5.3 L2 .eth Registry Controller

Similar to the registrar controller in ENS v1, the .eth registry controller will exist on L2 and facilitate registration and renewals of names by users. As in v1, this division exists in order to ensure registration and renewal logic, including pricing, can be updated, while allowing the .eth registry itself to be locked. Multiple controllers can be authorised by the DAO as needed.

4.5.4 L1 .eth Registry

The L1 .eth registry exists as an L1 counterpart to the L2 .eth registry described above. Its key responsibilities are:

- Maintaining expiration and ownership information for ejected .eth 2LDs
- Returning the fallback resolver for names not found in its own registry
- (Optionally) Facilitating L1 renewals of ejected names

The L1 .eth registry implements much of the same logic as the L2 .eth registry, with a controller pattern for registrations and renewals; however, this role is taken by the L1 ejection controller rather than a contract that allows direct registration.

One design decision yet to be determined is whether renewal of ejected names occurs on L2 or L1. If it occurs on L2, the L1 ejection controller is responsible for updating the L1 .eth registry with new expiration dates propagated from L2. Otherwise, a controller on the L1 .eth registry is responsible for renewing names natively and sending a message to L2 to update the L2 .eth registry on the change.

4.5.5 L1 .eth Fallback Resolver

This contract is set as the resolver on the L1 .eth registry, and has two key responsibilities:

1. For names not yet migrated to ENSv2, queries the legacy registry to find the resolver responsible for the name, and forwards resolution requests to that contract.
2. Uses CCIP-Read to fulfil resolution requests for names hosted on L2.

To determine which situation applies, the contract checks for existence of the name in the legacy registry. If the name has an expiration date in the future, and its owner is not the L1 migration contract, case 1 applies. Otherwise, case 2 applies.

When resolving names from L2, the fallback resolver effectively replicates the name resolution process, by submitting an EVM Gateway request that:

1. Recursively fetches registry information from the L2 `RegistryDatastore` contract until it finds the closest registry responsible for the parent of the target name.
2. Fetches the resolver address from that registry.
3. Fetches the requested data from that resolver.

Because of the nature of CCIP-Read, this all occurs via storage proofs. The desire to support wildcard resolution also means that step 3 needs to include several requests to fetch records first for an exact match, then for successively shorter suffixes, until a result is found.

4.5.6 Ejection Controllers

The ejection controllers are a pair of L1 and L2 contracts responsible for handling 'ejection' of names from L2 to L1. They each act as a controller on their corresponding .eth registry contract, and exchange messages between chains using cross-chain messaging.

The ejection process is as follows:

1. A user transfers the .eth 2LD on L2 to the L2 ejection controller, with calldata specifying the L1 owner address and a factory contract to create a new subregistry from.
2. The L2 ejection controller calls the L2 .eth registry, setting the name's subregistry address to `address(0)`.
3. The L2 ejection controller sends a cross-chain message to the L1 ejection controller, containing the L1 owner address, L1 subregistry address, and expiration date of the name.
4. The L1 ejection controller calls the L1 .eth registry, renewing the name until the L2 expiration time, and supplying the owner and subregistry address as specified.
5. The L1 .eth registry sets the subregistry address and transfers ownership of the name on L1 to the specified owner address.

   Moving a name back to L2 happens as follows:

6. A user transfers the .eth 2LD on L1 to the L1 ejection controller, with calldata specifying the L2 owner address and subregistry address.

7. The L1 ejection controller calls the L1 .eth registry to burn the name on L1.
8. The L1 ejection controller sends a cross-chain message to the L2 ejection controller, containing the L2 owner address and L2 subregistry address.
9. The L2 ejection controller transfers the L2 name, which was previously held by it, to the specified owner, and sets the subregistry address as required.

Note that both ejecting and reimporting names incurs a brief outage, during which the name will not resolve; this is necessary because cross-chain messaging is not instantaneous, while local updates are. This outage could be avoided during the ejection process by not zeroing out the L2 subregistry address, but this would introduce confusion as to which records are authoritative for an ejected name.

4.5.7 Migration Contracts

The migration contracts are a pair of L1 and L2 contracts responsible for handling migration from ENSv1 to ENSv2.

Initially, all existing .eth 2LDs will be owned by the L2 migration contract. When a user migrates their .eth 2LD to ENSv2, the following happens:

1. The user transfers their legacy .eth 2LD to the L1 migration contract, with calldata specifying the L2 owner and L2 subregistry to use. If the name is wrapped and locked, the registry must be the canonical user registry contract.
2. The L1 migration contract sends a message to the L2 migration contract, specifying name, owner, and subregistry contract.
3. The L2 migration contract updates the subregistry address for the name and transfers ownership of it to the specified owner address. If the original name was wrapped and locked, it locks the registry address for the name.

As a result of ownership of the name being transferred to the L1 migration controller, the L1 .eth fallback resolver will no longer treat the name as unmigrated. The L2 entries for unmigrated names will specify an interim subregistry and resolver, which instruct the L1 .eth fallback resolver to call the legacy L1 resolver for the name; this avoids an outage while the name is being migrated to L2.

Alternatively, the user can migrate the name to L1 as an ejected name. This entails the following process:

1. The user transfers their legacy .eth 2LD to the L1 migration contract, with calldata specifying an L1 owner and subregistry to use. If the name is wrapped and locked, the registry must be the canonical user registry contract.
2. The L1 migration contract sends a cross-chain message to the L2 migration contract, specifying that the name has been ejected.
3. The L2 migration contract transfers the L2 name to the L2 ejection authority contract.
4. The L1 migration contract also creates or updates the domain on the L1 .eth registry, setting its subregistry and transferring it to the desired owner. If the original name was wrapped and locked, it locks the registry address for the name.

Finally, a migration process exists by which owners of subnames of locked names (3LD and below, eg foo.locked.eth) can migrate their names even if the owner of the parent name has not yet migrated their name. This process follows the steps outlined above, with the exception that a shadow entry for the 2LD is created on L1 (and L2, if migrating to L2); this entry is transferred to the migration authority.

4.5.8 User Registry

A default implementation is provided of a user registry contract. This is a general-purpose contract that facilitates commonly desired behaviour such as creation and assignment of subdomains, locking names, etc. For most users, this will be the only registry implementation they need to use.

The user registry contract also contains functionality required to replicate the functionality permitted by Name Wrapper fuses. This includes the ability to lock the registry for subnames, and to prevent changes to key values such as the resolver, expiration, etc, for the name the registry is concerned with.

When a user migrates a Locked name to ENSv2, the registry is set to the User Registry, and this is locked in the parent .eth registry so that the user cannot change it.

Finally, the user registry implements optional functionality that requires creation of a new subname include a proof that the same name does not exist on L1. This functionality is required in order to migrate a locked L1 name to L2, to prevent the owner of the locked name from creating subnames on L2 that shadow existing names on L1.

4.5.9 User Resolver

A new user resolver based on the current public resolver will be developed. This offers effectively the same functionality, but has access control settings that allow its current owner to set all records. Each user will have their own instance of the user resolver, rather than using a single common public resolver. This provides for a simpler access model, as well as removing the risk of obsolete records from previous owners being shown to users after a domain has been transferred.

4.5.10 Proxy Factory Contracts

Since both the user registry and user resolver contracts are intended to be instantiated repeatedly - either per-user (for the resolver) or per-name (for the registry), minimising the cost of doing this is valuable. A proxy factory implementation will be developed that can be used to instantiate either of these contracts by deploying a minimal proxy that references a standard implementation.

Aside from cheap instantiation, these factories will also provide an easy way to identify whether a contract is an instance of the factory. The proxies will also permit users to upgrade the implementation to a new version.

5. Roadmap

We anticipate the following milestones for development of ENSv2:

1. Development of a new Universal Resolver interface compatible with both ENSv1 and ENSv2, along with a proxy contract that can be upgraded one time only by the DAO.
2. Update client libraries and applications to use the new Universal Resolver standard in place of the legacy resolution method.
3. Development and release of an initial PoC-quality revision of the new ENSv2 contracts.
4. Selection of an L2 platform.
5. Deployment of completed ENSv2 contracts to testnets.
6. DAO approval and deployment of ENSv2 contracts to mainnet and L2.
7. Migration of existing data to L2.

8. Switchover to new ENSv2 contracts and deprecation of old contracts.

6. Conclusion

6.1 Anticipated Impact

We anticipate the deployment of ENSv2 will represent a step change in functionality and usability for both developers and end-users. Developers will benefit from the increased flexibility provided by the new registry design, as well as being able to benefit from other infrastructure deployed as part of the migration. Users will benefit from the reduced transaction fees and increased throughput that comes from hosting their names on an L2, while still being able to choose to retain the security and availability guarantees of hosting their name on L1 if desired.

6.2 Future Directions

We anticipate the release of ENSv2 to unlock a new set of possibilities centred around easily customisable domain ownership; allowing users to provide custom registry implementations unlocks a plethora of new applications, while preserving more flexibility than the name wrapper provided for.